

2008-11-13

New alloy treatment

The API for creation and initialization of alloy models changed in the following way:

- `PhysicalModelInterface::init()` is **only** called for non-alloy models, therefore also `read_database()` and `do_init()`. `read_database()` should not read parameters used exclusively in alloys.
- The following methods have been renamed:

- `PhysicalModelInterface::build_alloy()` to `PhysicalModelInterface::init_alloy()`
- `virtual read_bowing_parameters()` to `virtual PhysicalModelInterface::read_database_alloy()`
- `virtual PhysicalModelInterface::calculate_VCA()` to `virtual PhysicalModelInterface::do_init_alloy()`

`init_alloy()` calls `read_database_alloy()` and `do_init_alloy()`

`do_init_alloy()` has to perform all necessary initialization similar to `do_init()` (it is perfectly acceptable to call `do_init()` from `do_init_alloy()` if this makes sense and `do_init()` does not create submodels)

- `PhysicalModelInterface::copy()` is **not anymore** intended to create an identical copy of a model but rather an instance of the same model class that will be functional after a call to `init()` or `init_alloy()`. Therefore, `copy_from()` does not anymore need to copy all members and in most cases does not need to be implemented, **unless your model contains data members that cannot be set up during** `do_init()` / `do_init_alloy()`
- as `do_init()` is not called for an alloy, you have to create submodels inside of `do_init_alloy()`. For this purpose there are two helper methods in `PhysicalModelInterface` to assist you in this task:
 - `T* create_submodel_copy(const T* other)` creates a copy of `other` by calling `copy()` and sets the alloy material reference in the new object. You have to call `init_alloy()` on the new object.
 - `T* create_submodel_alloy(const T* A, const T* B, double xa)` creates a copy of `A` by means of `create_submodel_copy()` and calls `init_alloy()` on the new object.

Use `create_submodel_copy()` in cases where you have to do additional initialization from outside before calling `init_alloy()`.

2008-11-14

New database access

The use of `(get_material()->get_database()).get_data_file()` together with `GetPot` is deprecated. You can now extract data directly from the database.

The material files are organized in sections which are marked by a header of the form `[sectionname]`. If there are global data that are not to be put into a section, they necessarily have to be before the first section header (or after an empty header `[ ]`). The use of the latter is discouraged from.

The database is accessed as follows from inside of a `PhysicalModelInterface` derived class:

```
Database& db = get_database(); // member of
                                // PhysicalModelInterface

db.set_section("pippo");
your_variable = db.get("variablename", default_value);

vector<X> some_vector;
db.get("variablename", some_vector);

vector<vector<X> > some_matrix;
db.get("variablename", some_matrix);
```

Some tips:

- initialize members in the constructor and use the variable itself as default value
- if there's no sense in having a default value then you can require it to be present in the database by using a variant of `get()` with `true` as third argument. In this case `get()` will throw an exception if the variable cannot be found in the database.
- Data read from the database can be overridden in `do_init()`. For this purpose, use `get_parameter()` instead of `get_options().get_option()` as the former is looking also in the Region section of the input file.